LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 07 – ABSTRACT & INTERFACE



NAMA: Nur Tsalits Alwan Mubarok

NIM: 103132400039

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1) Abstract Class

Abstract class adalah kelas yang dideklarasikan dengan kata kunci abstract dan tidak dapat diinstansiasi menjadi objek. Kelas ini hanya dapat digunakan dengan cara diturunkan, dan kelas turunannya yang dapat dibuat menjadi objek. Abstract class dapat memiliki atribut, konstruktor, method biasa yang sudah berisi implementasi, dan abstract method. Abstract method adalah method yang hanya memiliki deklarasi tanpa isi, sehingga wajib di-override oleh kelas turunan konkretnya. Abstract class digunakan untuk merepresentasikan konsep umum, misalnya Employee sebagai konsep umum dari SalariedEmployee.

2) Interface

Interface mendefinisikan kumpulan method atau konstanta yang harus diimplementasikan oleh kelas lain. Kelas yang menggunakan interface menggunakan kata kunci implements (disebut realization). Atribut dalam interface otomatis bersifat public static final (konstanta), interface tidak boleh memiliki konstruktor, dan semua method bersifat public dan abstract secara default. Berbeda dengan kelas yang hanya boleh memiliki satu induk, sebuah kelas dapat mengimplementasikan lebih dari satu interface (multiple inheritance).

3) Default Method dan Static Method (Java 8)

Sejak Java 8, interface dapat memiliki method yang sudah berisi implementasi.

- Default method (default) dapat langsung digunakan oleh kelas implementasi tanpa harus di-override. Fitur ini memungkinkan pengembang menambahkan method baru ke interface tanpa memengaruhi kelas yang sudah ada.
- Static method (static) adalah method milik interface itu sendiri, dipanggil lewat nama interface (misalnya MyInterface.anotherNewMethod()), dan tidak dapat di-override oleh kelas implementasi.

4) Perbedaan Abstract Class dan Interface

Abstract Class	**Interface**
Boleh memiliki method dengan implementasi	Pada dasarnya method tanpa body (kecuali default dan static)
Dapat mendeklarasikan variabel biasa	Hanya dapat mendefinisikan konstanta
Merupakan hasil turunan dari kelas induk	Tidak punya hubungan inheritance langsung dengan kelas tertentu
Tidak mendukung multiple inheritance	Mendukung multiple inheritance

5) Polimorfisme

Polimorfisme adalah kemampuan objek dari kelas berbeda untuk diperlakukan sebagai satu tipe yang sama, lalu merespons pemanggilan method yang sama

dengan caranya masing-masing. Dengan interface, objek dari hierarki yang berbeda dapat disimpan dalam satu array bertipe interface dan diproses dalam satu perulangan.

B. Guided

1. Abstract Class

```
//file Employee.java

package guided1;

public abstract class Employee {
   private String name;

   public Employee(String name) {
      this.name = name;
   }

   public String getName() {
      return name;
   }

   public abstract double earnings();
}
```

```
//file Salaried.java

package guided1;

public class SalariedEmployee extends Employee {
   private double weeklySalary;

   public SalariedEmployee(String name, double weeklySalary) {
      super(name);
      this.weeklySalary = weeklySalary;
   }

   @Override
   public double earnings() {
      return weeklySalary;
   }
}
```

```
//file Main.java

package guided1;
```

```
public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Budi", 50000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output:

```
Nama: Budi
Gaji: Rp50000.0
```

2. Interface Payable

```
//file Employee.java

package guided2;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
//file Payable.java

package guided2;

public interface Payable {
    double getPaymentAmount();
}
```

```
//file Invoice.java

package guided2;

public class Invoice implements Payable {
   private String partName;
   private int quantity;
   private double pricePerItem;

   public Invoice(String partName, int quantity, double pricePerItem){
      this.partName = partName;
      this.quantity = quantity;
      this.pricePerItem = pricePerItem;
   }

   public String getPartName(){
      return partName;
   }

   @Override
   public double getPaymentAmount(){
      return quantity * pricePerItem;
   }

}
```

```
//file SalariedEmployee.java

package guided2;

public class SalariedEmployee extends Employee {
   private double weeklySalary;

   public SalariedEmployee(String name, double weeklySalary) {
      super(name);
      this.weeklySalary = weeklySalary;
   }

   @Override
   public double earnings() {
      return weeklySalary;
   }
}
```

```
//file Main.Java
```

```
package guided2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 15000);
        items[1] = new SalariedEmployee("Budi", 50000);

        for (Payable item : items) {
            System.out.println("Jumlah yang perlu dibayar: Rp" + item.getPaymentAmount());
        }
    }
}
```

Output:

```
Jumlah yang perlu dibayar: Rp30000.0
Jumlah yang perlu dibayar: Rp50000.0
```

3. Interface Java 8

```
//file Employee.java

package guided3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
//file Invoice.java

package guided3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }

}
```

```
//file Payable.java

package guided3;

public interface Payable {
    double getPaymentAmount();

    //fitur baru
    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    //Fungsi utilitas
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

```
//file SalariedEmployee.java

package guided3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

```
//file Main.java

package guided3;

public class Main {
    public static void main(String[]args){
        Payable.logTransaction("Sitem Pembayaran diinisialisasi");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("RAMR DDR 5 16 GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 500000);

        for (Payable item : items){
            item.printReceipt();
        }
        Payable.logTransaction("Transaksi selesai diproses.");
    }
}
```

Output:

```
[AUDIT LOG] Sitem Pembayaran diinisialisasi
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp500000.0
[AUDIT LOG] Transaksi selesai diproses.
```

C. Unguided

1) Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli Tiket Nonton, tetapi juga bisa membeli makanan/minuman (Paket Snack F&B).
   Secara konsep sistem, Tiket Bioskop dan Paket Snack berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.
   Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (interface), dan fitur modern Java 8+

1) Jawaban:

```
//file ItemTransaksi.java

package unguided;

public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();

    // Default method: format baris struk
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" + getNominalBayar());
    }

    // Static method: utilitas log kasir
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

```
//file TiketBioskop.java

package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
```

```
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    // Kontrak publik kasir, hanya meneruskan hasil logika internal tiket
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

```
//file TiketReguler.java

package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return nomorStudio;
    }

    @Override
```

```
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

```
//file TiketVIP.java

package unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi()
                + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")";
    }
}
```

```
//file PaketSnack.java

package unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
```

```
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return namaPaket;
    }

    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi +
")";
    }
}
```

```
//file Main.java

package unguided;

public class Main {
    public static void main(String[] args) {
        // 1. Uji proteksi abstraksi (tidak boleh di-instansiasi)
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);

        // 2. Buka kasir
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");
```

```
        // 3. Buat objek transaksi
        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack snack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1,
55000);

        // 4. Array polimorfik heterogen
        ItemTransaksi[] keranjang = {tiketReguler, tiketVIP, snack};

        // 5. Pemrosesan dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup kasir
        System.out.println("------------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("==========================================================
===========");
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output:

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan:

1) ItemTransaksi.java (Interface)
   Interface ini menjadi kontrak umum bagi semua barang di kasir, dengan dua method abstrak: getNominalBayar() dan getNamaTransaksi(). Di dalamnya ada default method cetakRincianItem() yang mencetak satu baris struk dengan

memanggil kedua method kontrak tadi, dan static method catatLogKasir() untuk mencetak log sistem.

2) TiketBioskop.java (Abstract Class)
   Kelas induk tiket yang implements ItemTransaksi. Atributnya (judulFilm, hargaDasar, jumlahKursi) bersifat private dan diakses lewat getter. Kelas ini mendeklarasikan abstract method hitungTotalBayar() dan getNamaTransaksi(). Method getNominalBayar() sudah diimplementasikan di sini dan hanya mengembalikan hasil hitungTotalBayar().
3) TiketReguler.java dan TiketVIP.java (Concrete Subclass)
   Keduanya turunan TiketBioskop dan memanggil super(...) untuk mengisi atribut induk. TiketReguler menghitung total dengan hargaDasar * jumlahKursi, sedangkan TiketVIP dengan (hargaDasar + biayaFasilitasVIP) * jumlahKursi. Format nama transaksi di-override sesuai jenis tiket.
4) PaketSnack.java (Concrete Class)
   PaketSnack.java (Concrete Class)
   Kelas yang tidak berasal dari hierarki tiket, tetapi mengimplementasikan ItemTransaksi secara langsung. Total bayarnya dihitung dari jumlahPorsi * hargaPerPorsi.
5) Main.java
   Pembuatan objek TiketBioskop langsung dijadikan komentar karena abstract class tidak dapat diinstansiasi. Ketiga objek (TiketReguler, TiketVIP, PaketSnack) disimpan dalam array ItemTransaksi[]. Perulangan for-each memanggil cetakRincianItem() dan menjumlahkan getNominalBayar() menjadi grand total. Ini contoh polimorfisme, karena satu perintah yang sama menghasilkan keluaran berbeda tergantung jenis objeknya.

Kesimpulan:

1) Abstract class berfungsi sebagai kerangka dasar bagi kelas-kelas turunan. Kelas ini tidak dapat diinstansiasi dan memaksa kelas turunan mengimplementasikan abstract method-nya.
2) Interface berfungsi sebagai kontrak yang dapat diterapkan pada kelas dari hierarki berbeda. Pada unguided, TiketBioskop dan PaketSnack berasal dari silsilah yang berbeda tetapi dapat diproses bersama karena sama-sama mengimplementasikan ItemTransaksi.
3) Default method dan static method pada Java 8 memungkinkan interface diberi fitur baru tanpa mengubah kelas yang sudah mengimplementasikannya.
4) Kombinasi abstract class, interface, dan polimorfisme membuat program lebih fleksibel dan mudah dikembangkan, karena item baru cukup mengimplementasikan interface tanpa mengubah logika perulangan di Main.
5) Perbedaan utamanya: abstract class cocok untuk hubungan “adalah sebuah” (is-a) dengan atribut dan perilaku bersama, sedangkan interface cocok untuk mendefinisikan kemampuan yang bisa dimiliki kelas mana pun.